

**Corso di
Architettura degli Elaboratori
a.a. 2025/2026**

**Il livello logico digitale:
Circuiti logici digitali di base**

Livello della logica digitale

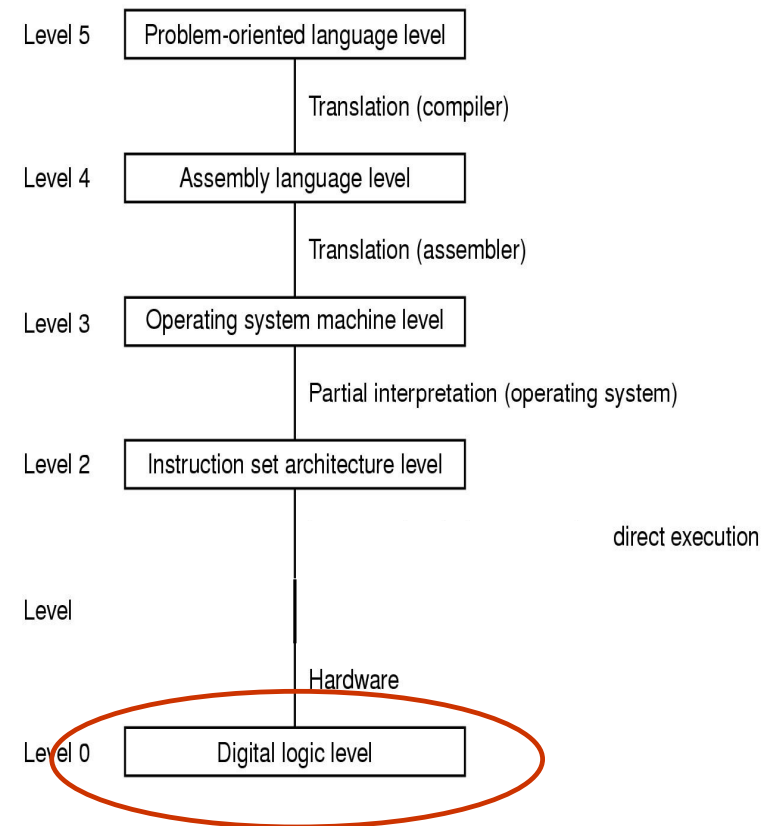
Livello della *Logico-Digitale*

Costituenti di base del computer:

- porte
- registri
- memoria

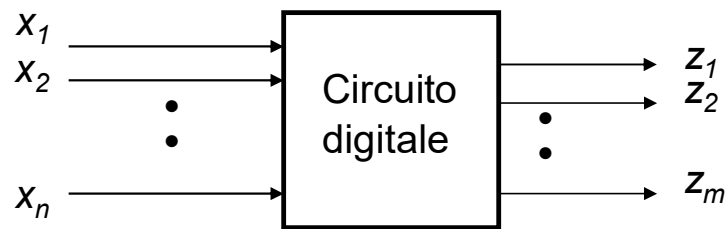
Sotto questo livello ci sono i dispositivi

(funzionamento interno delle porte: transistor)



Circuiti digitali

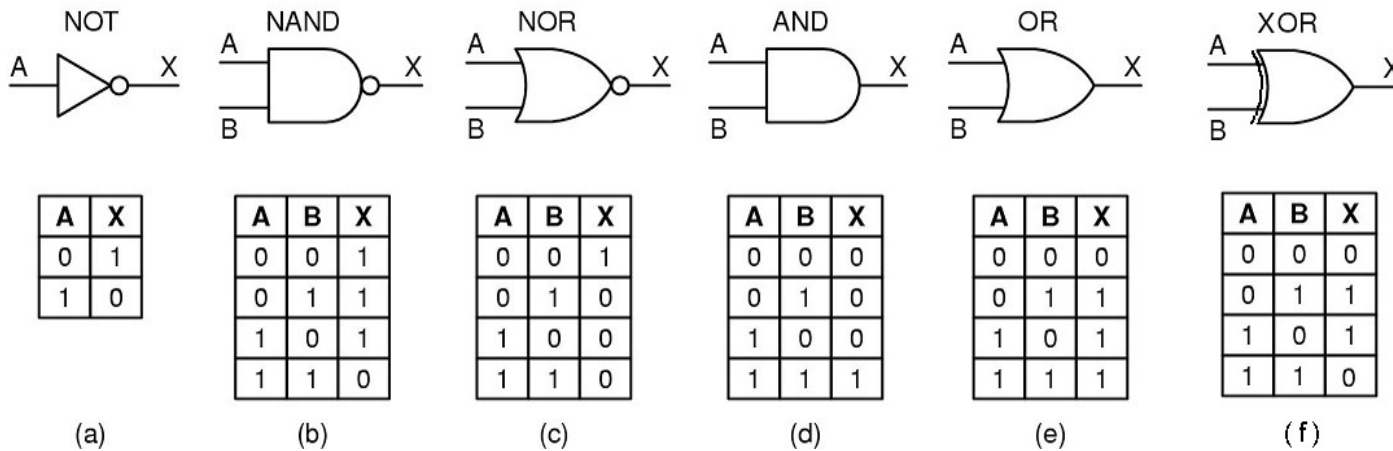
- Gli elementi di base con cui si sono costruiti i calcolatori si chiamano **circuiti digitali** (o reti digitali)
- Sono dispositivi che utilizzano **solo due valori logici**:
0 (segnale tra 0 e 1 volt) e 1 (segnale tra 2 e 5 volt).
 - I valori di tensione possono anche essere altri.
- Un circuito digitale trasforma segnali (binari) di **ingresso** $x_1, x_2, \dots, x_n$ nei segnali (binari) di **uscita** $z_1, z_2, \dots, z_m$.



Porte logiche

- I circuiti sono detti **combinatori** quando l'uscita è funzione esclusivamente dell'ingresso; sono detti **sequenziali** quando l'uscita è funzione oltre che dell'ingresso anche di uno **stato**.
- Gli elementi primitivi dei circuiti digitali sono chiamati **porte logiche** e calcolano alcune funzioni di questi segnali a due valori.
- Questi dispositivi si basano sul fatto che si può far funzionare un **transistor** come un interruttore binario molto veloce.

Porte logiche



I circuiti possono essere descritti con un linguaggio per la descrizione di hardware, ad esempio, in Verilog:

```
module not_gate(
    input A,
    output X
);

    assign X = ~A;
endmodule
```

```
module nor_gate(
    input A,
    input B,
    output X
);

    assign X = ~(A | B);
endmodule
```

```
module and_gate(
    input A,
    input B,
    output X
);

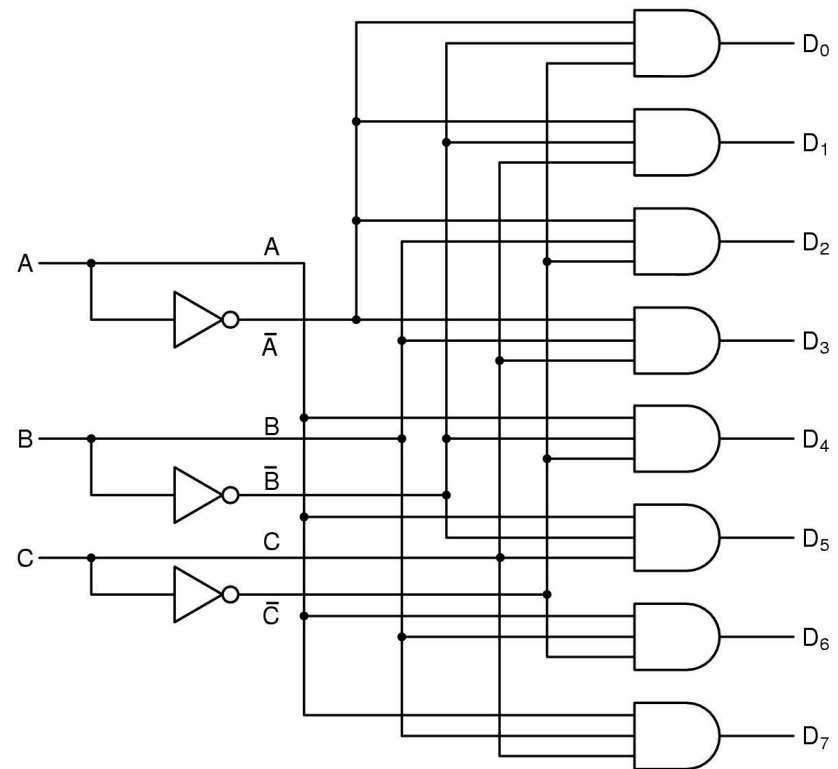
    assign X = A & B;
endmodule
```

```
module xor_gate(
    input A,
    input B,
    output X
);

    assign X = A ^ B;
endmodule
```

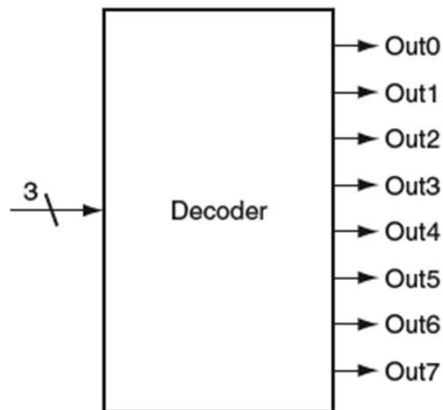
Circuiti combinatori

- Circuito combinatorio: l'output viene **determinato solo dagli input**
- Convenzione: l'incrocio tra due linee non implica alcuna connessione a meno che non sia presente il simbolo • nel punto di intersezione



Circuiti combinatori: decoder

- **Decoder:** prende un numero di n bit come ingresso e lo usa per selezionare (mettere a 1, asserire) una delle 2^n linee di uscita
- Può essere utilizzato per attivare una certa componente (vedi ALU più avanti), oppure un banco di memoria, ecc.
- Interpretiamo gli ingressi A B C (o I2 I1 I0) come le cifre di un numero in base 2 con A (I2) quella più significativa

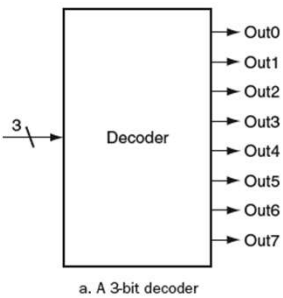


a. A 3-bit decoder

Inputs			Outputs							
I2	I1	I0	Out7	Out6	Out5	Out4	Out3	Out2	Out1	Out0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

b. The truth table for a 3-bit decoder

Circuiti combinatori: decoder



Inputs			Outputs							
12	11	10	Out7	Out6	Out5	Out4	Out3	Out2	Out1	Out0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

b. The truth table for a 3-bit decoder

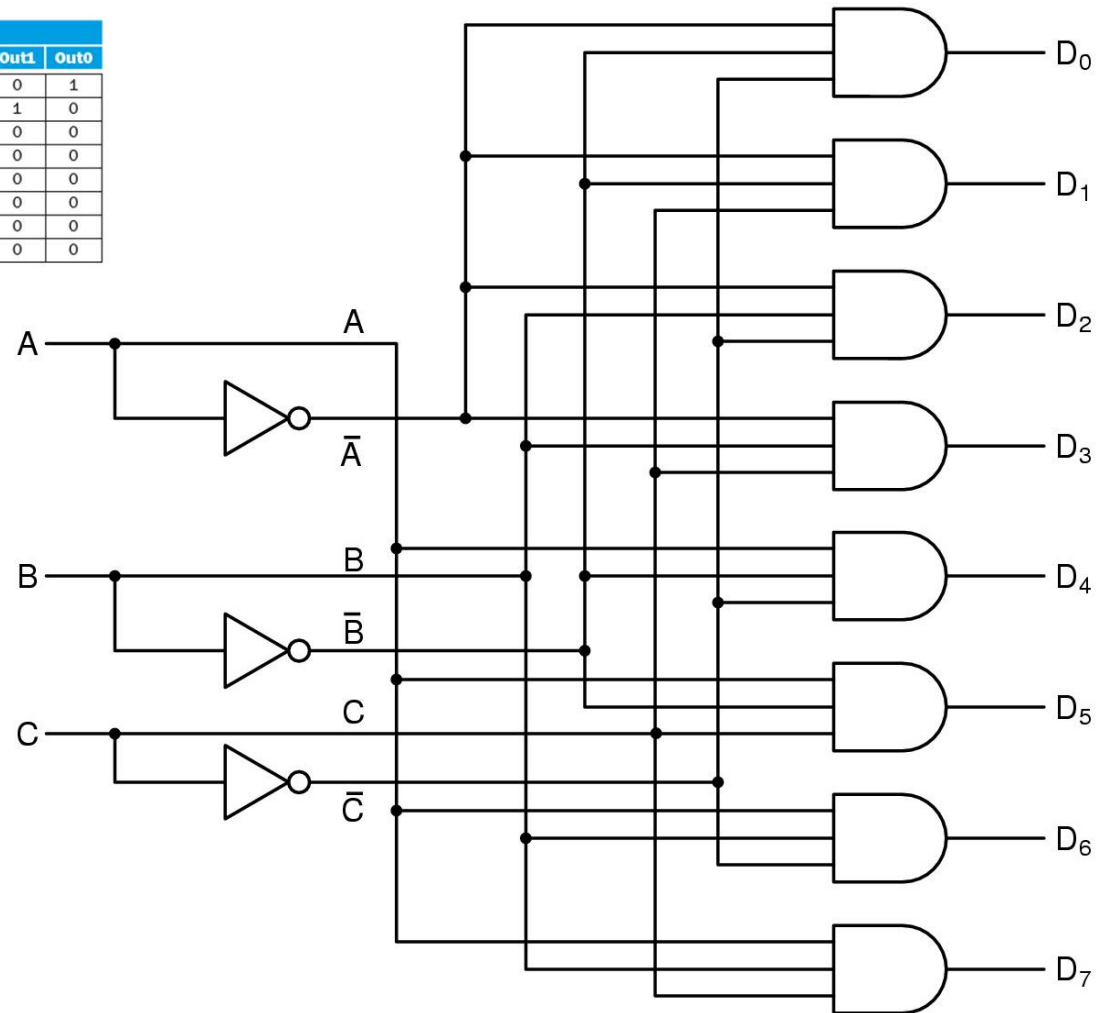
```

module decoder_3to8(
  input  [2:0] in, // 3-bit input
  output [7:0] out // 8-bit output
);

  assign out[0] = (in == 3'b000);
  assign out[1] = (in == 3'b001);
  assign out[2] = (in == 3'b010);
  assign out[3] = (in == 3'b011);
  assign out[4] = (in == 3'b100);
  assign out[5] = (in == 3'b101);
  assign out[6] = (in == 3'b110);
  assign out[7] = (in == 3'b111);

endmodule

```



Circuiti combinatori: multiplexer

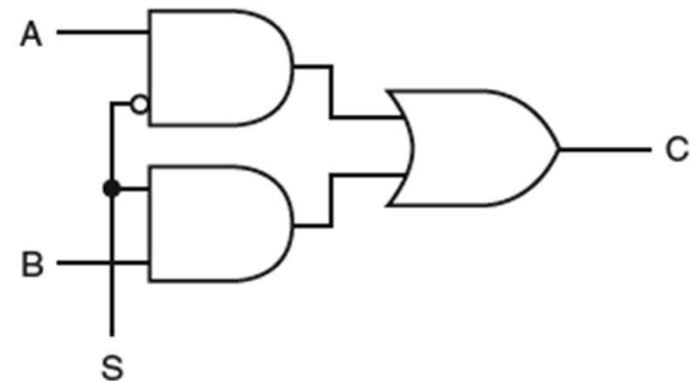
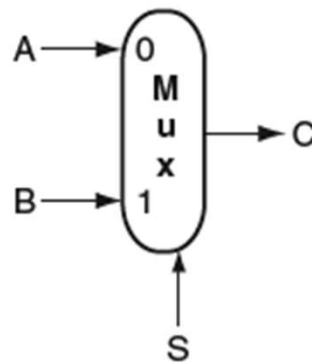
- **Multiplexer (o selettore):**

2 ingressi, 1 uscita e 1 ingresso di controllo

- La linea di controllo determina quale dei 2 ingressi deve essere selezionato per essere inviato all'uscita

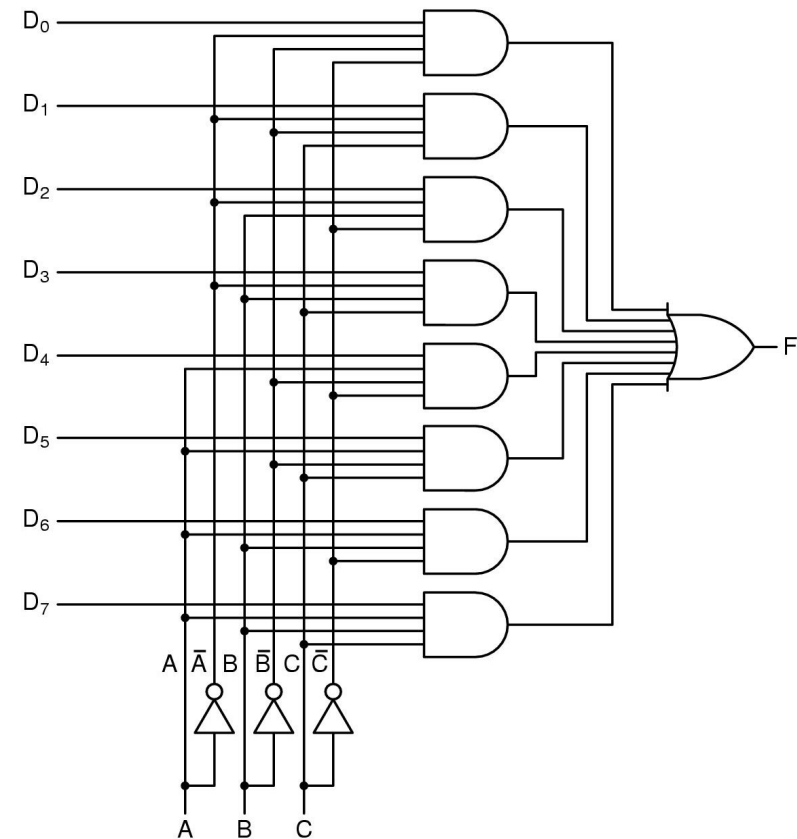
- Esempio con 2 ingressi (A e B), 1 uscita (C) ed 1 ingresso di controllo (S)

```
module mux_1bit(  
    input A,        // primo input  
    input B,        // secondo input  
    input S,        // controllo  
    output C         // uscita  
);  
    // Se S=0, C=A  
    // Se S=1, C=B  
    assign C = (~S & A) | (S & B);  
endmodule
```



Circuiti combinatori: multiplexer

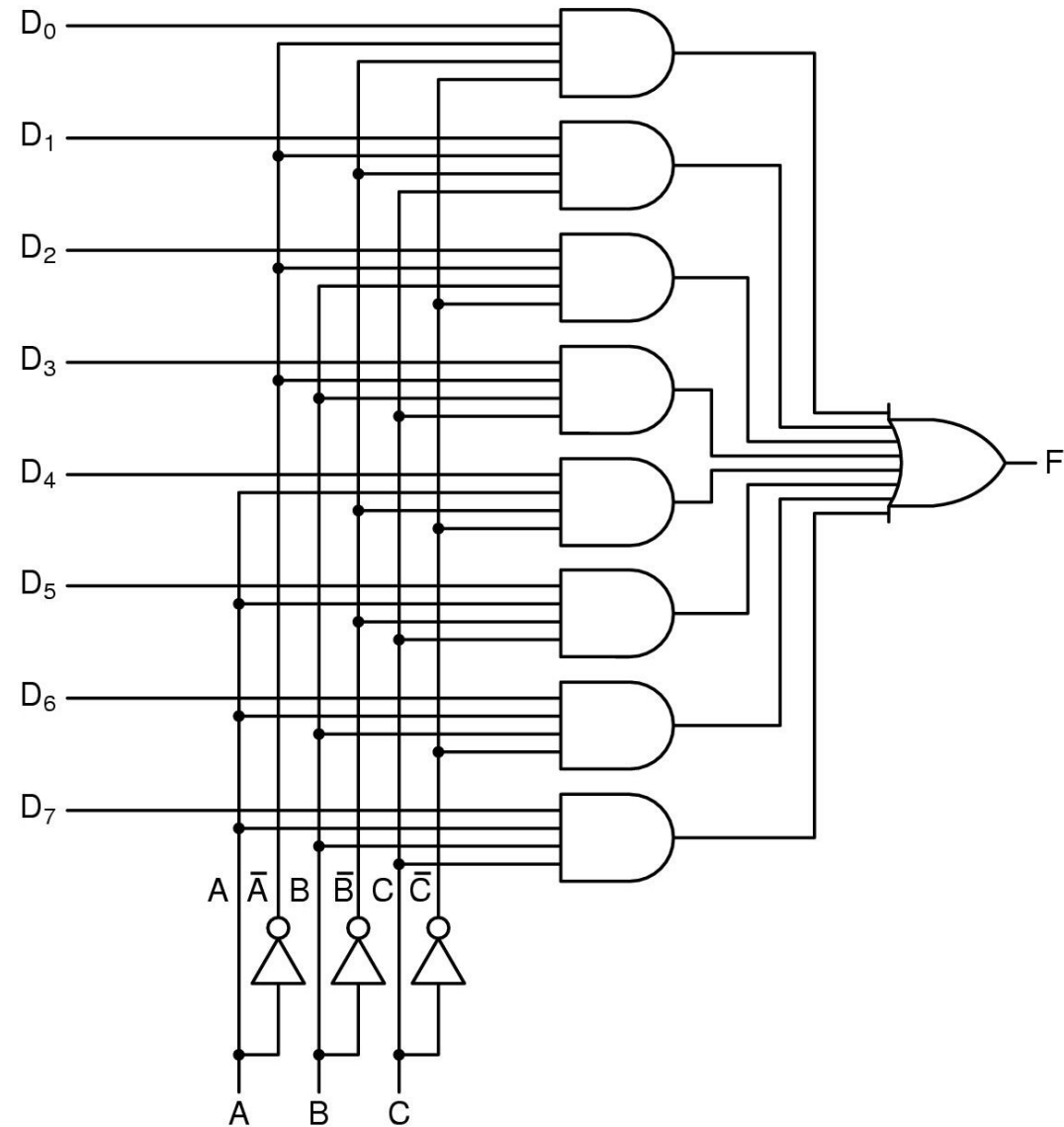
- **Multiplexer:** In generale, possiamo avere 2^n ingressi, 1 uscita e n ingressi di controllo
- Le linee di controllo determinano quale dei 2^n ingressi deve essere selezionato per essere inviato all'uscita
- Esempio con 8 ingressi ($D_0, \dots, D_7$), 1 uscita (F) e 3 ingressi di controllo (A, B e C)



Multiplexer

Un multiplexer è composto da un decoder dove ogni AND accoglie un ingresso e l'OR finale

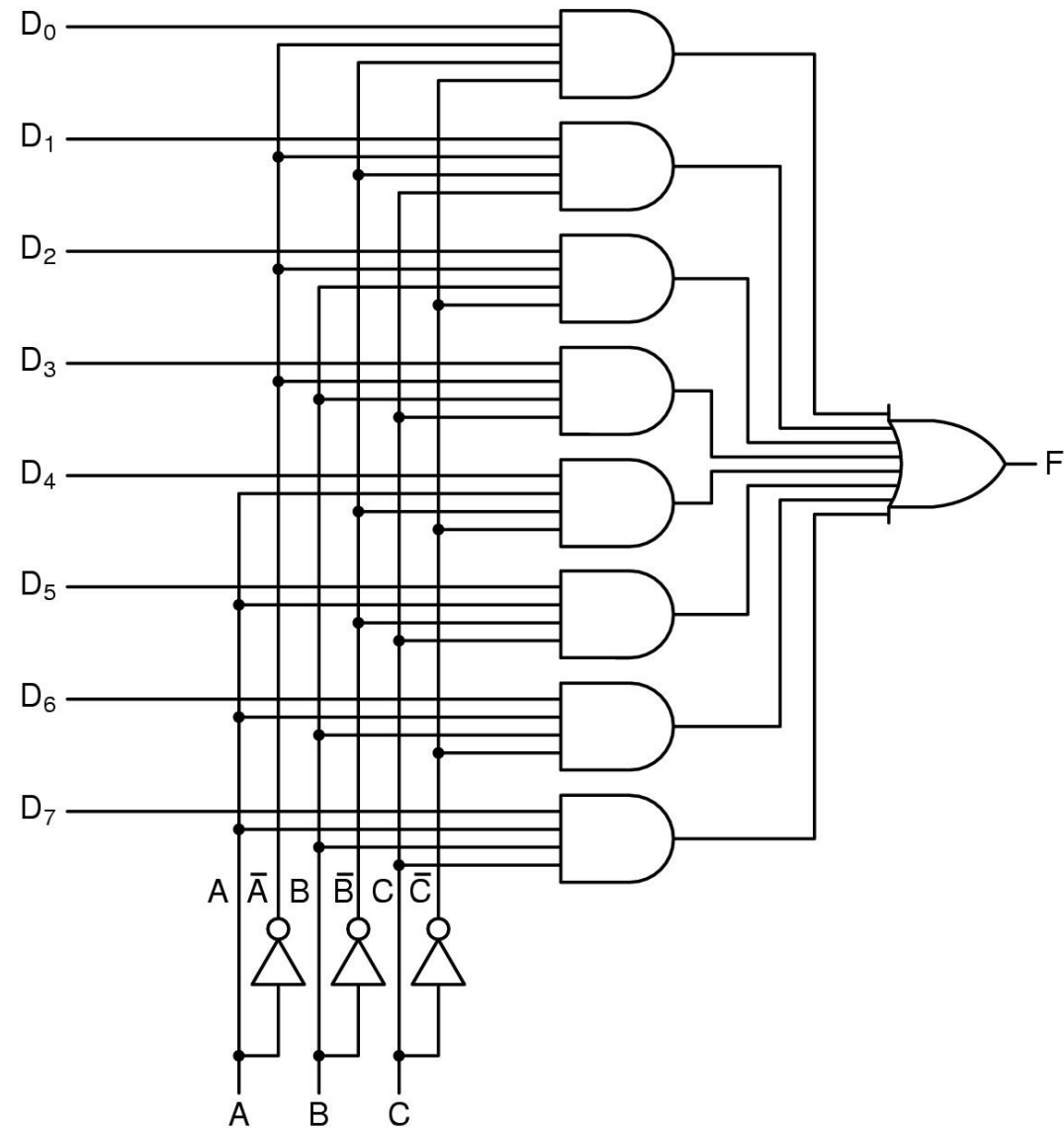
```
module mux_8to1(  
    input [7:0] D,      // 8 ingressi di dati (D0-D7)  
    input A, B, C,      // Ingressi di selezione  
    output F            // Uscita  
);  
  
    assign F = (D[0] & ~A & ~B & ~C) |  
               (D[1] & ~A & ~B & C) |  
               (D[2] & ~A & B & ~C) |  
               (D[3] & ~A & B & C) |  
               (D[4] & A & ~B & ~C) |  
               (D[5] & A & ~B & C) |  
               (D[6] & A & B & ~C) |  
               (D[7] & A & B & C);  
  
    // ogni termine rappresenta:  
    // => (dato) AND (uscita di decoder corrispondente)  
    // L'OR finale combina tutti questi termini  
  
endmodule
```



Multiplexer

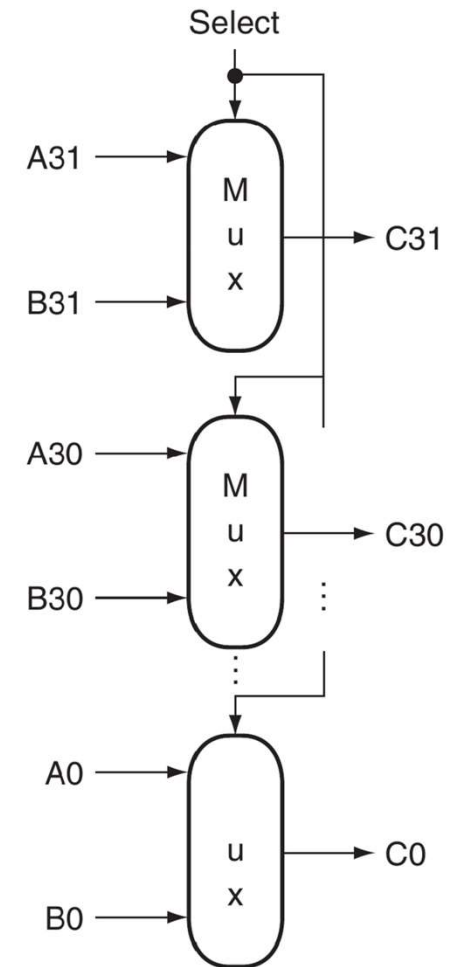
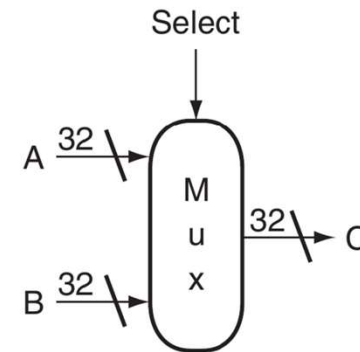
Un multiplexer è composto da un decoder dove ogni AND accoglie un ingresso e l'OR finale

```
module mux_8to1(  
    input [7:0] D,      // 8 ingressi di dati (D0-D7)  
    input A, B, C,      // Ingressi di selezione  
    output F            // Uscita  
);  
    wire [2:0] sel;      // segnale di selezione  
    assign sel = {A, B, C}; // combina A, B, C  
  
    // se sel=000, F=D0  
    // se sel=001, F=D1  
    // ecc.  
    assign F = (sel == 3'b000) ? D[0] :  
                (sel == 3'b001) ? D[1] :  
                (sel == 3'b010) ? D[2] :  
                (sel == 3'b011) ? D[3] :  
                (sel == 3'b100) ? D[4] :  
                (sel == 3'b101) ? D[5] :  
                (sel == 3'b110) ? D[6] :  
                (sel == 3'b111) ? D[7];  
  
endmodule
```



Circuiti combinatori: multiplexer

- **Multiplexer (o selettore)**: 2 ingressi da 32 line ciascuno, 1 uscita da 32 linee e 1 ingresso di controllo
- La linea di controllo determina quale dei 2 ingressi da 32 linee deve essere selezionato per essere inviato all'uscita da 32 linee
- Esempio con 2 ingressi (A e B), 1 uscita (C) ed 1 ingresso di controllo (Select)



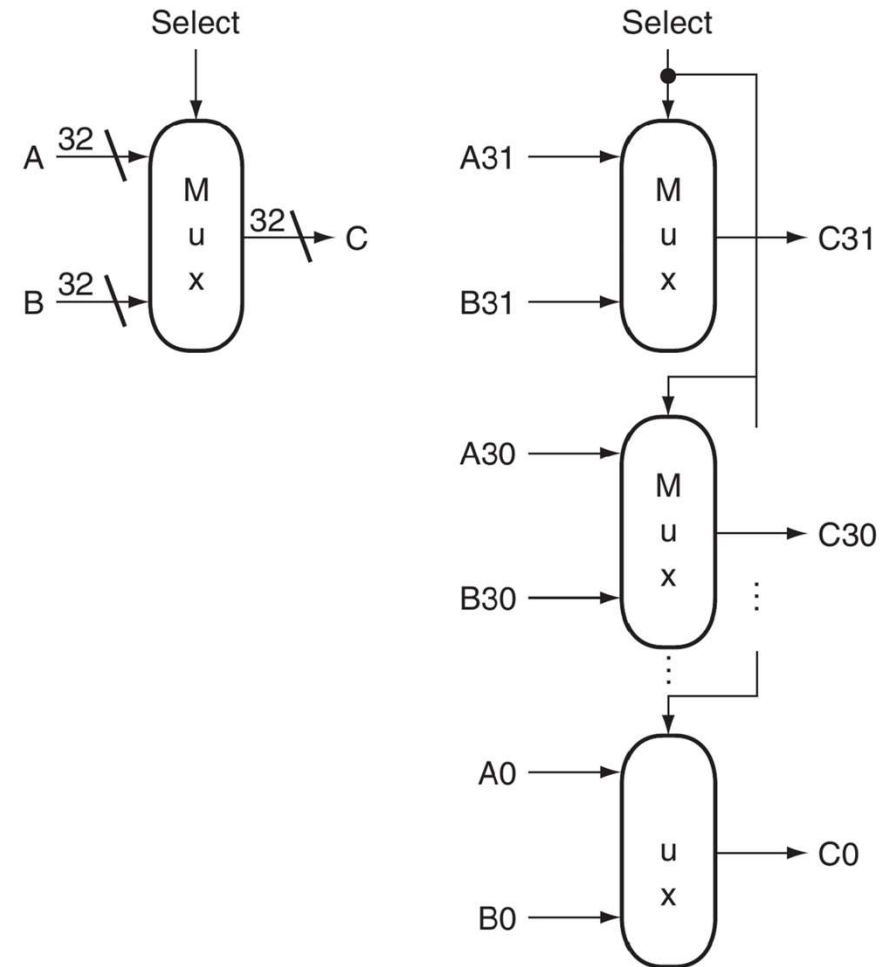
Circuiti combinatori: multiplexer

```
// Multiplexer 2-a-1 a 32 bit (esplicito)
module mux_32bit(
    input [32:0] A,      // Primo ingresso da 32bit (A31...A0)
    input [32:0] B,      // Secondo ingresso da 32bit (B31...B0)
    input S,             // Ingresso di controllo
    output [32:0] C      // Uscita da 32 bit (C31...C0)
);
    // Istanziamento esplicito dei primi multiplexer a 1 bit
    // Multiplexer per il bit 0 (come mostrato nella figura)
    mux_1bit mux_bit0 (
        .A(A[0]),
        .B(B[0]),
        .S(S),
        .C(C[0])
    );

    // Multiplexer per il bit 1
    mux_1bit mux_bit1 (
        .A(A[1]),
        .B(B[1]),
        .S(S),
        .C(C[1])
    );

    // Per i rimanenti bit (2-31)
    // si potrebbe continuare con l'istanziamento esplicita

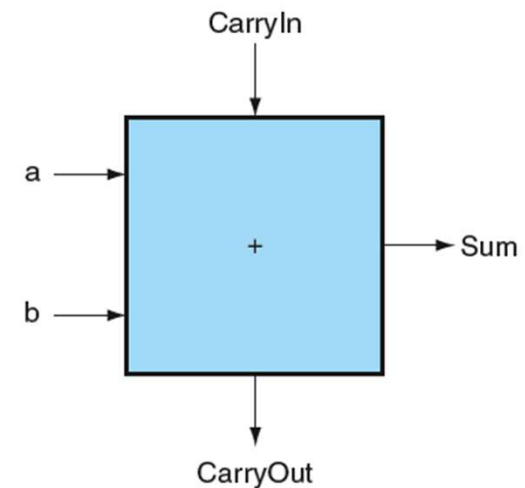
    // o usiamo un'assegnazione semplice direttamente
    // assign C = S ? B : A;
endmodule
```



Circuiti numerici: addizionatori

- riceve in ingresso due bit (cifre in base 2) da sommare, **a** e **b** nello schema
- riceve in ingresso un bit (cifra in base 2) di riporto, **CarryIn** nello schema
- restituisce un bit (cifra in base 2) in uscita che rappresenta il risultato, **Sum** nello schema
- restituisce un bit (cifra in base 2) in uscita che rappresenta il riporto, **CarryOut** nello schema

Inputs			Outputs	
a	b	CarryIn	CarryOut	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

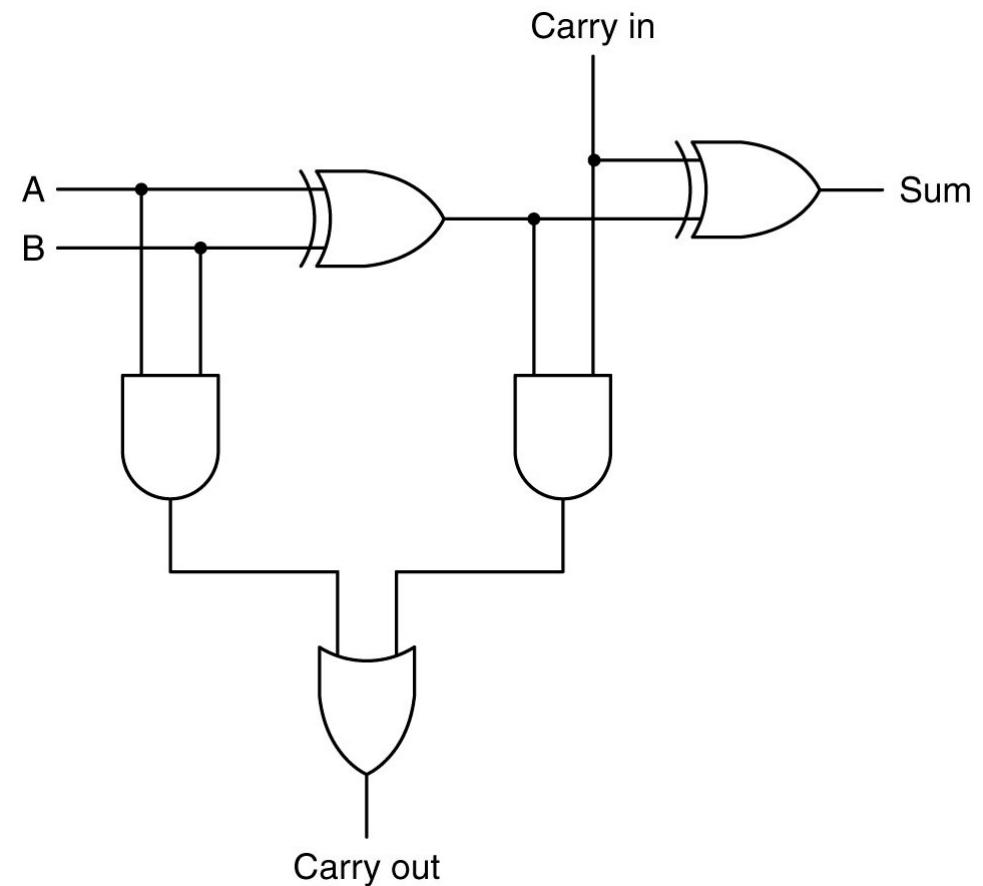


Circuiti numerici: addizionatori con XOR

$$C_{out} = C_{In}(a \text{ XOR } b) + a b$$

$$Sum = (a \text{ XOR } b) \text{ XOR } C_{In}$$

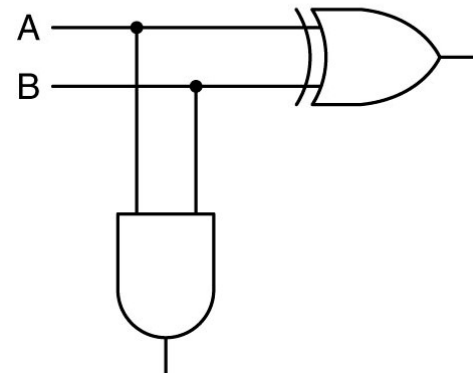
Inputs			Outputs	
a	b	CarryIn	CarryOut	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Circuiti numerici: addizionatori con XOR

```
// Modulo half adder
// Sum = a XOR b
// Carry = a AND b
module half_adder(
    input a,          // Primo bit di ingresso
    input b,          // Secondo bit di ingresso
    output sum,       // Bit di somma
    output carry      // Bit di riporto
);
    // Calcola la somma
    xor xor1(sum, a, b);

    // Calcola il riporto
    and and1(carry, a, b);
endmodule
```



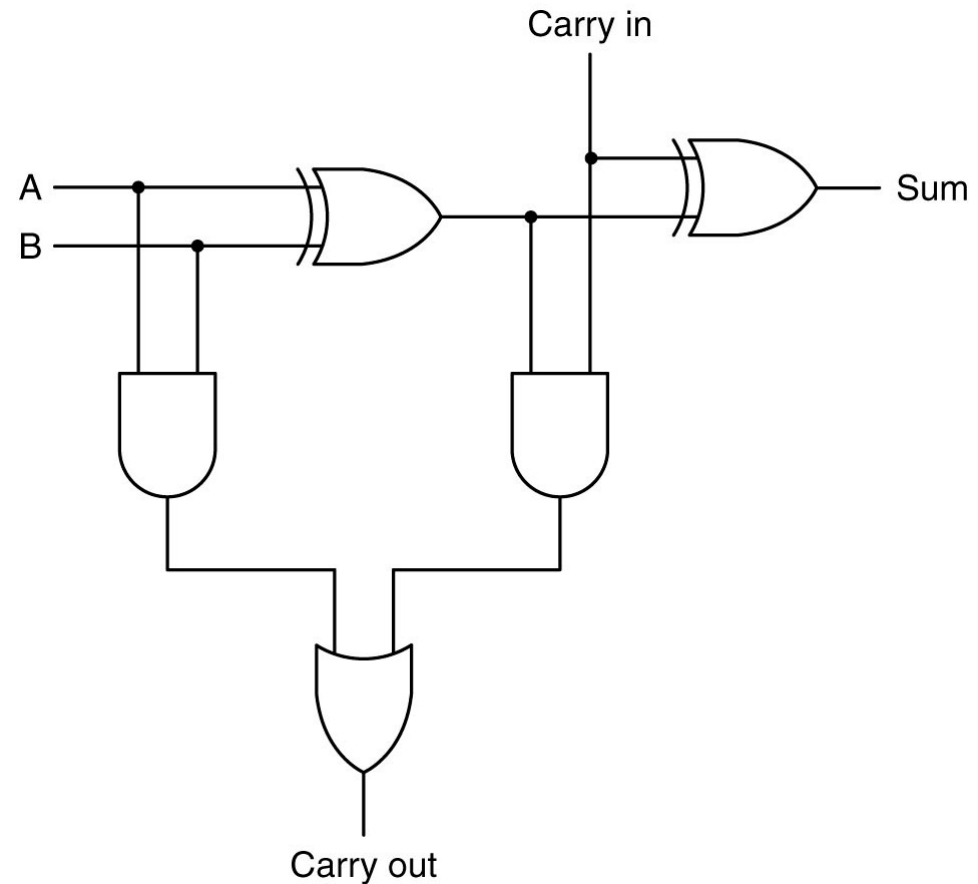
Circuiti numerici: addizionatori con XOR

```
// Circuito Addizionatore a 1-bit utilizzando due half adder
// Half adder 1: calcola a + b
// Half adder 2: calcola (a + b) + carry_in
// I due carry vengono combinati per produrre carry_out
module adder(
    input a,           // Bit di ingresso a
    input b,           // Bit di ingresso b
    input carry_in,    // Bit di riporto in ingresso
    output sum,        // Bit di somma in uscita
    output carry_out   // Bit di riporto in uscita
);
    wire sum_half1;    // Risultato della somma del primo half adder
    wire carry_half1;  // Risultato del carry del primo half adder
    wire carry_half2;  // Risultato del carry del secondo half adder

    // Primo half adder: calcola a + b
    half_adder half1(
        .a(a),
        .b(b),
        .sum(sum_half1),
        .carry(carry_half1)
    );

    // Secondo half adder: calcola (a + b) + carry_in
    half_adder half2(
        .a(sum_half1),
        .b(carry_in),
        .sum(sum),
        .carry(carry_half2)
    );

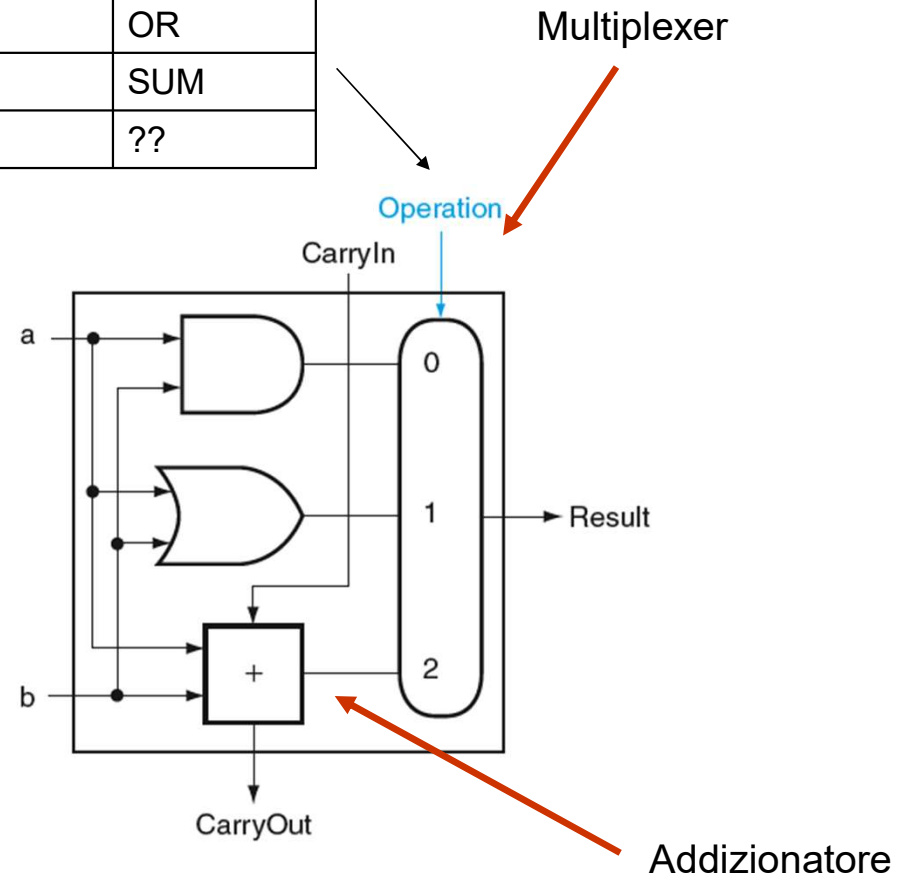
    // Combina i due carry per ottenere carry_out
    or1(carry_out, carry_half1, carry_half2);
endmodule
```



RISC-V: ALU ad 1 bit

- Dati **a** e **b** è in grado di calcolare:
 - Result** = **a** AND **b**
 - Result** = **a** OR **b**
 - Result** = **a** + **b** (somma)
- Un ingresso di controllo (**Operation**) seleziona l'operazione desiderata. Dovendo selezionare 3 possibili ingressi in realtà abbiamo 2 linee **Operation** di controllo.
- Un terzo ingresso fornisce in **CarryIn** per la somma
- Una seconda uscita rappresenta il **CarryOut**

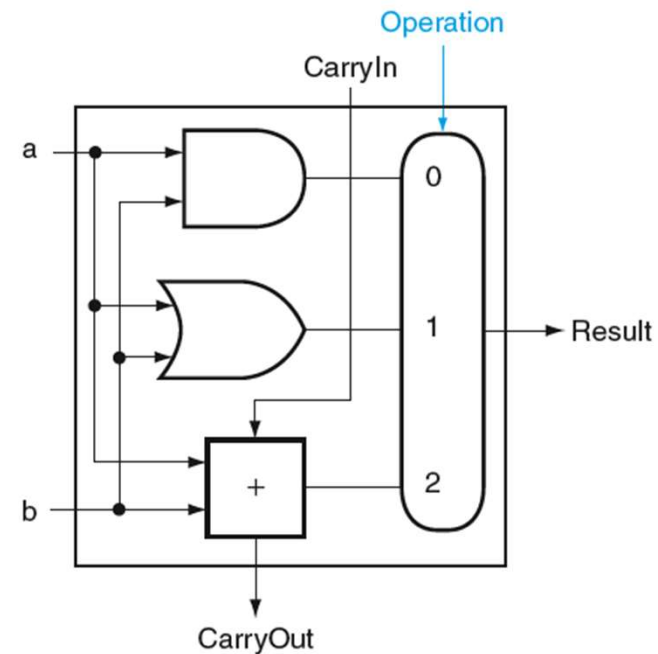
Operation	Function
00	AND
01	OR
10	SUM
11	??



RISC-V: ALU ad 1 bit

Operation	Function
00	AND
01	OR
10	SUM
11	??

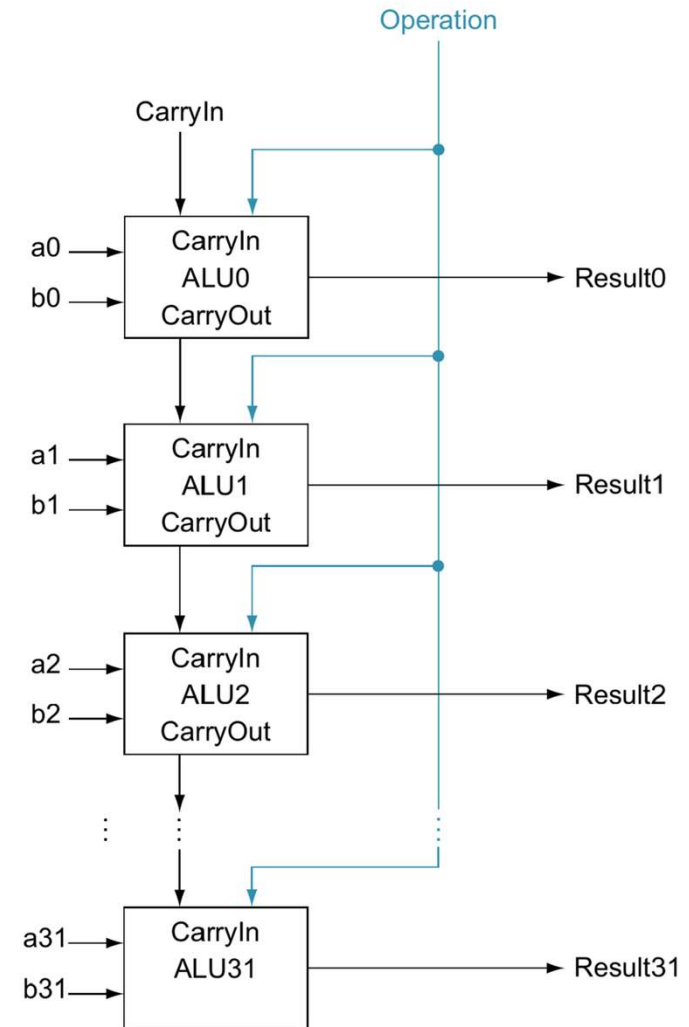
```
module alu_1bit(  
    input a, b,           // Ingressi operandi  
    input [1:0] operation, // Selettore operazione  
    input carryin,        // Riporto in ingresso  
    output result,        // Risultato dell'operazione  
    output carryout        // Riporto in uscita  
);  
  
    // Fili interni per i segnali intermedi  
    wire and_out, or_out; // Uscite delle porte logiche  
    wire sum;             // Risultato dell'addizione  
  
    // Operazioni logiche di base  
    assign and_out = a & b;  
    assign or_out  = a | b;  
  
    // Operazione di somma e calcolo del riporto  
    assign {carryout, sum} = a + b + carryin;  
  
    // Multiplexer per selezionare l'operazione finale  
    assign result = (operation == 2'b00) ? and_out :  
                    (operation == 2'b01) ? or_out :  
                    (operation == 2'b10) ? sum :  
                    (operation == 2'b11) ? 1'b0 : 1'b0;  
  
endmodule
```



RISC-V: ALU a 32 bit

- Dati **a** e **b** su 32 bit è in grado di calcolare:
 - Result** = **a** **AND** **b** (bit a bit)
 - Result** = **a** **OR** **b** (bit a bit)
 - Result** = **a** + **b** (somma su 64 cifre)
- Un ingresso di controllo (**Operation**) seleziona l'operazione desiderata. Dovendo selezionare 3 possibili ingressi in realtà abbiamo 2 linee **Operation** di controllo.

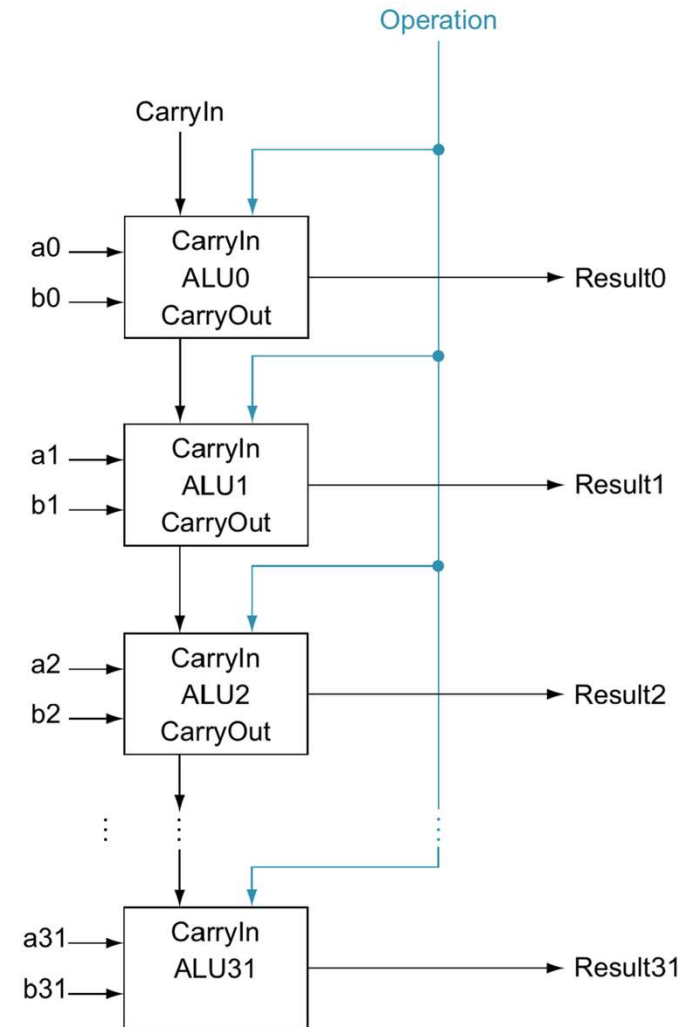
Operation	Function
00	AND
01	OR
10	SUM
11	??



RISC-V: ALU a 32 bit

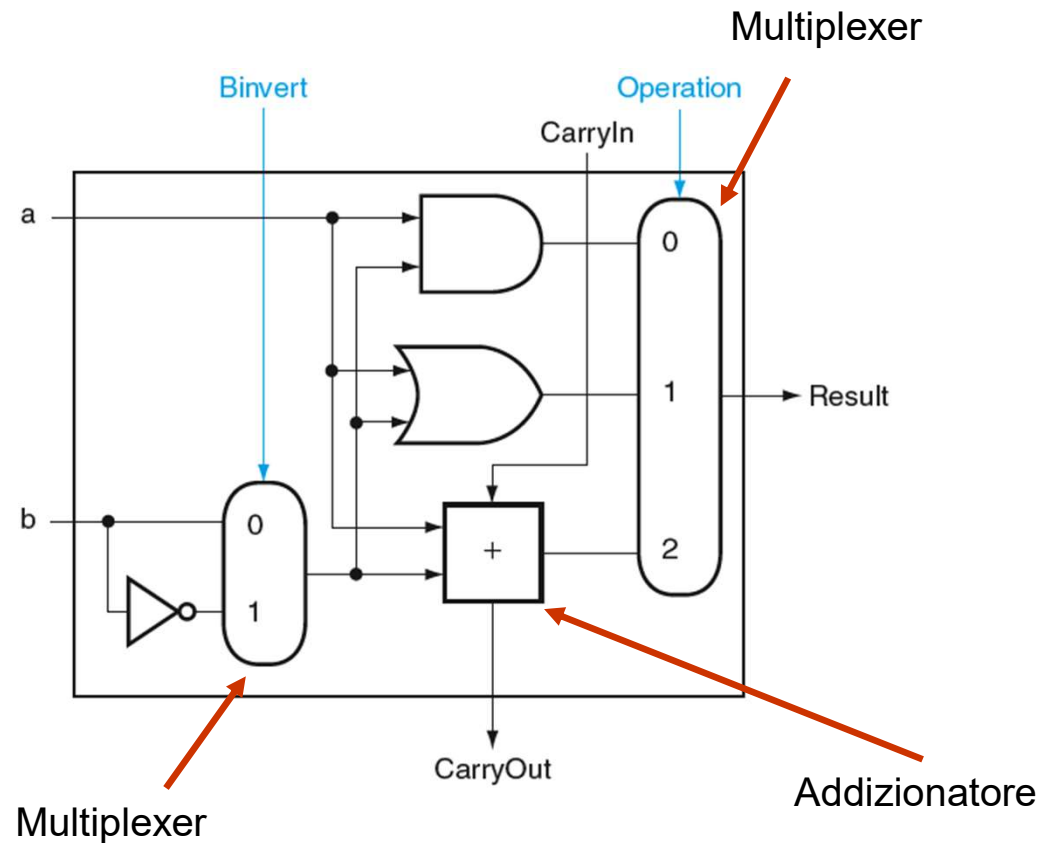
- Una ALU per operandi **a** e **b** ad n bit si ottiene utilizzando n ALU ad 1 bit
- Ogni ALU esegue l'operazione sulla coppia di bit degli operandi **a** e **b** nella stessa posizione (bit slice)
- Per sommare due operandi **a** e **b** di n bit il **CarryOut** dell'ALU per il bit in posizione i diventa il CarryIn del bit in posizione i + 1
- Il **CarryIn** del bit meno significativo (a0 e b0) può essere usato come segnale di incremento per calcolare **a + b + 1** (servirà per le sottrazioni)

Operation	Function
00	AND
01	OR
10	SUM
11	??



RISC-V: ALU ad 1 bit (sub)

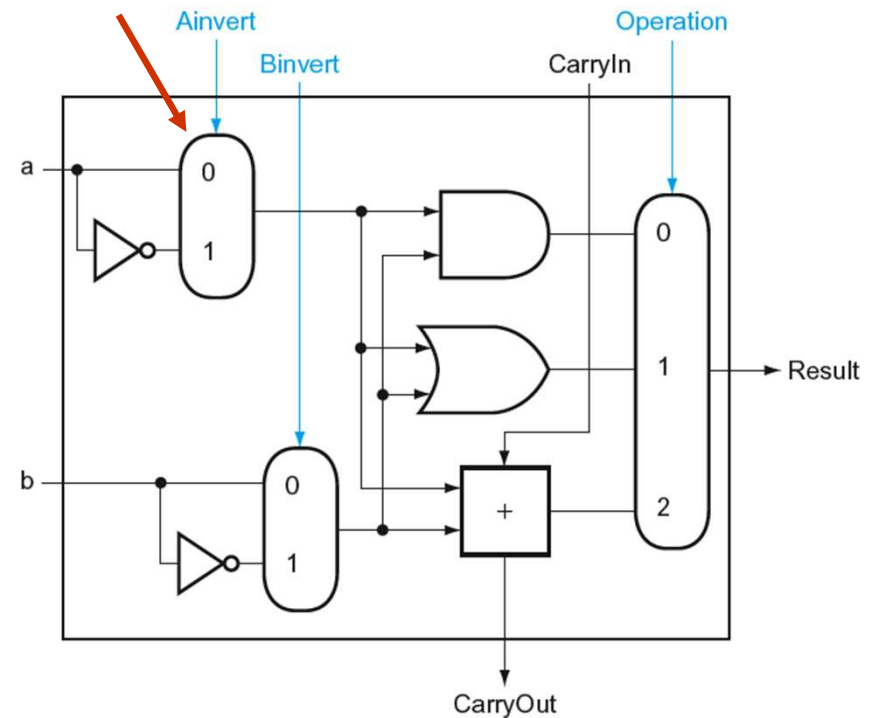
- Dati **a** e **b** è in grado di calcolare le funzioni precedenti con **b** o $\bar{b}$
- Un ingresso di controllo aggiuntivo (**Binvert**) seleziona l'operazione desiderata
- Serve, ad esempio, per le sottrazioni (**a - b**)
 - Ricordare il complemento a 2
 - Con l'uso di $\bar{b}$
 - **CarryIn = 1** per il bit meno significativo



RISC-V: ALU ad 1 bit (nor)

- Dati **a** e **b** è in grado di calcolare le funzioni precedenti con **a** o $\bar{a}$
- Un ingresso di controllo aggiuntivo (**Ainvert**) seleziona l'operazione desiderata
- Serve, ad esempio, per funzioni NOR: $\overline{(a \text{ OR } b)}$ con l'uso di $(\bar{a} \text{ AND } \bar{b})$

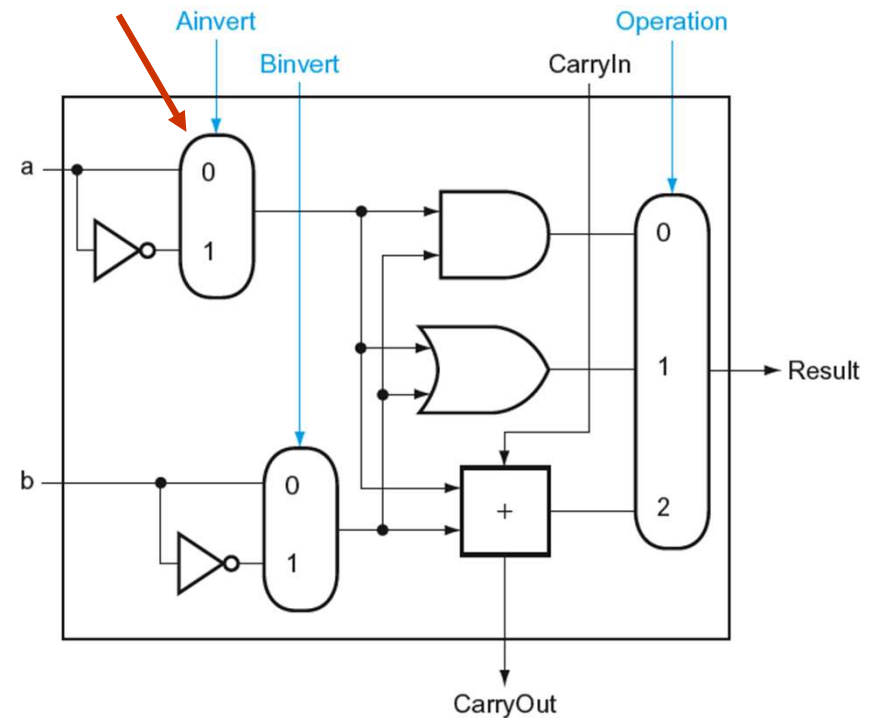
Multiplexer



RISC-V: ALU ad 1 bit (nor)

```
module alu_1bit(  
    input a, b,           // Ingressi operandi  
    input ainvert,        // Controlla l'inversione di a  
    input binvert,        // Controlla l'inversione di b  
    input [1:0] operation, // Selettore operazione  
    input carryin,        // Riporto in ingresso  
    output result,        // Risultato dell'operazione  
    output carryout       // Riporto in uscita  
);  
  
    // Fili interni per i segnali intermedi  
    wire a_int, b_int; // Valori di a e b dopo l'eventuale inversione  
    wire and_out, or_out; // Uscite delle porte logiche  
    wire sum;           // Risultato dell'addizione  
  
    // Multiplexer per l'inversione degli ingressi  
    assign a_int = ainvert ? ~a : a;  
    assign b_int = binvert ? ~b : b;  
  
    // Operazioni logiche di base  
    assign and_out = a_int & b_int;  
    assign or_out = a_int | b_int;  
  
    // Operazione di somma e calcolo del riporto  
    assign {carryout, sum} = a_int + b_int + carryin;  
  
    // Multiplexer per selezionare l'operazione finale  
    assign result = (operation == 2'b00) ? and_out :  
                    (operation == 2'b01) ? or_out :  
                    (operation == 2'b10) ? sum :  
                    (operation == 2'b11) ? 1'b0 : 1'b0;  
  
endmodule
```

Multiplexer



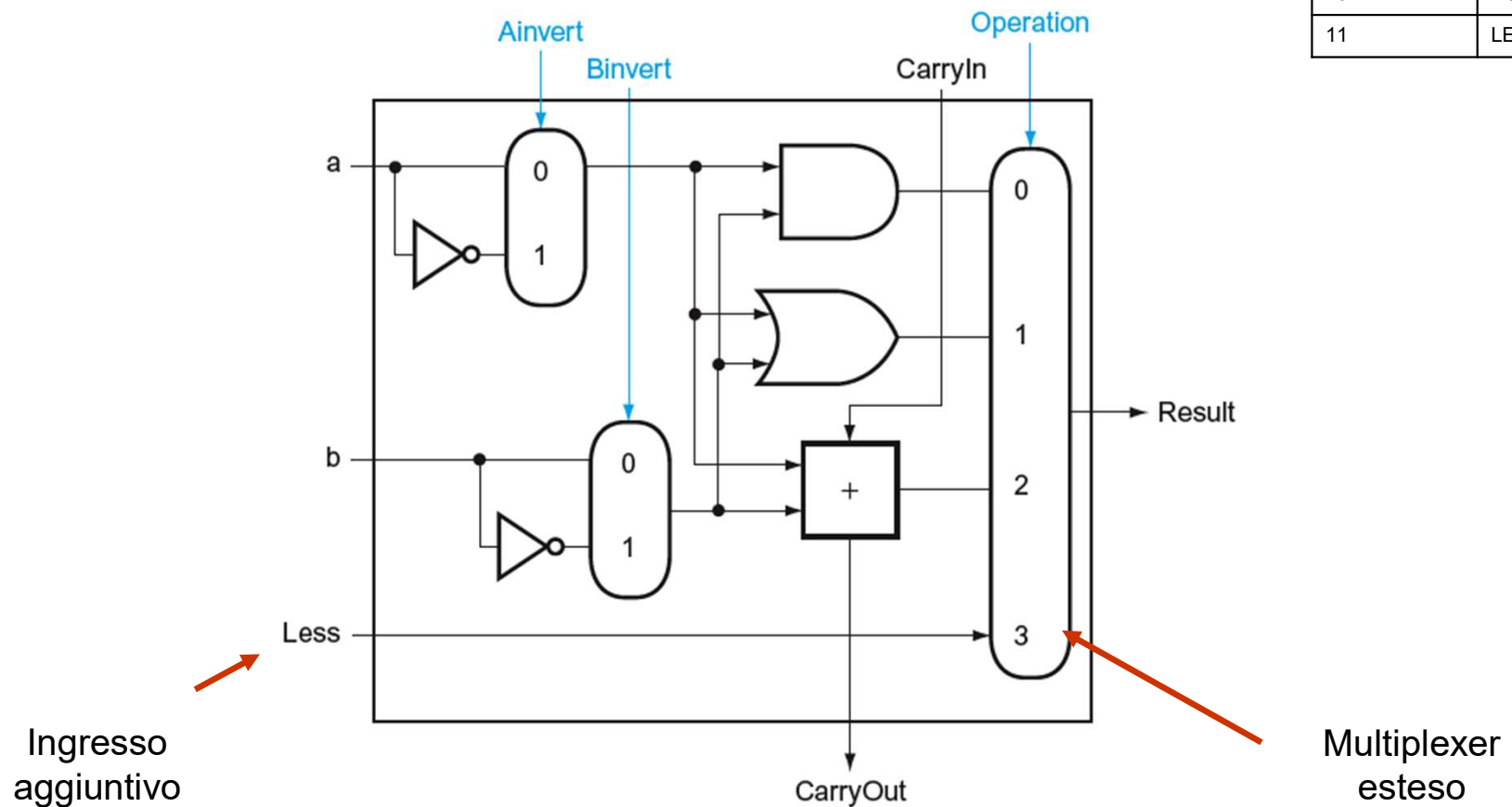
RISC-V: ALU per slt

- L'istruzione **Set on Less Than (slt)** restituisce 1 se $rs1 < rs2$ e 0 altrimenti
 - Il valore di tutti i bit in uscita tranne quello meno significativo deve essere 0
 - Il valore del bit meno significativo dipende dal confronto
-
- Idea: **usare la sottrazione**
 $(a - b) < 0$ implica $a < b$
 - Per vedere se $(a - b)$ è **negativo** verifico se il **bit più significativo vale 1**

RISC-V: ALU per slt

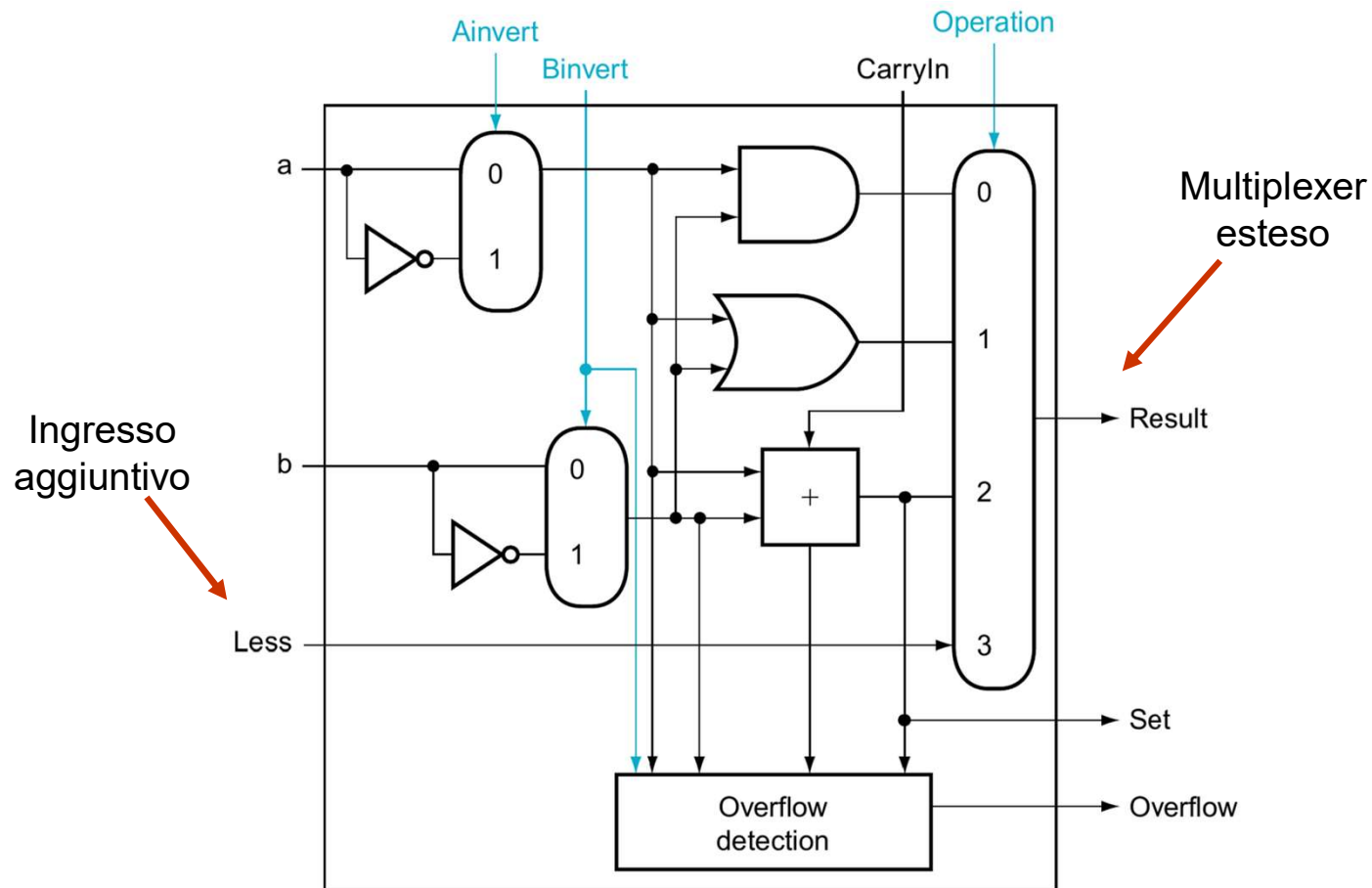
ALU estesa per ognuno dei 31 bit meno significativi degli operandi

Operation	Function
00	AND
01	OR
10	SUM
11	LESS



RISC-V: ALU per slt

ALU estesa per il 32-esimo bit, quello più significativo degli operandi



RISC-V: ALU per slt

31 bit meno significativi

ALU estesa per il 32-esimo bit

```
module alu_lbit_type1(
    input a, b,           // Ingressi operandi
    input ainvert,        // Controlla l'inversione di a
    input binvert,        // Controlla l'inversione di b
    input [1:0] operation, // Selettore operazione
    input carryin,        // Riporto in ingresso
    input less,           // Ingresso aggiuntivo per operazione "LESS"
    output result,        // Risultato dell'operazione
    output carryout       // Riporto in uscita
);

// Fili interni per i segnali intermedi
wire a_int, b_int; // Valori di a e b dopo l'eventuale inversione
wire and_out, or_out; // Uscite delle porte logiche
wire sum; // Risultato dell'addizione

// Inversione degli ingressi
assign a_int = ainvert ? ~a : a;
assign b_int = binvert ? ~b : b;

// Operazioni logiche di base
assign and_out = a_int & b_int;
assign or_out = a_int | b_int;

// Operazione di somma e calcolo del riporto
assign {carryout, sum} = a_int + b_int + carryin;

// Multiplexer esteso per selezionare l'operazione finale
// Nota: ora include l'ingresso "less" per l'operazione 11
assign result = (operation == 2'b00) ? and_out :
                (operation == 2'b01) ? or_out :
                (operation == 2'b10) ? sum :
                (operation == 2'b11) ? less : 1'b0;

endmodule
```

```
module alu_msb(
    input a, b,           // Ingressi operandi (bit più significativo)
    input ainvert,        // Controlla l'inversione di a
    input binvert,        // Controlla l'inversione di b
    input [1:0] operation, // Selettore operazione
    input carryin,        // Riporto in ingresso
    input less,           // Ingresso aggiuntivo per operazione "LESS"
    output result,        // Risultato dell'operazione
    output carryout,      // Riporto in uscita
    output set,           // Uscita "Set" per comparazione minore di
    output overflow       // Segnalazione di overflow
);

// Fili interni per i segnali intermedi
wire a_int, b_int; // Valori di a e b dopo l'eventuale inversione
wire and_out, or_out; // Uscite delle porte logiche
wire sum; // Risultato dell'addizione

// Inversione degli ingressi
assign a_int = ainvert ? ~a : a;
assign b_int = binvert ? ~b : b;

// Operazioni logiche di base
assign and_out = a_int & b_int;
assign or_out = a_int | b_int;

// Operazione di somma e calcolo del riporto
assign {carryout, sum} = a_int + b_int + carryin;

// Set è il bit di segno del risultato dell'addizione (usato per SLT)
assign set = sum;

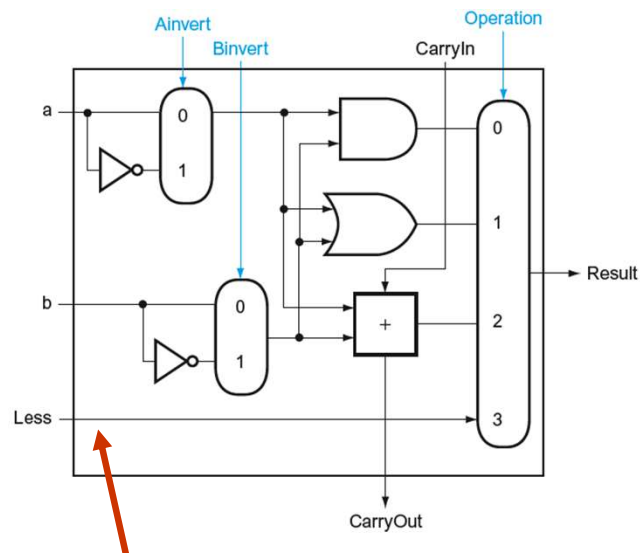
// gli operandi hanno lo stesso segno, il risultato ha segno opposto
assign overflow = (a_int == b_int) && (sum != a_int);

// Multiplexer esteso per selezionare l'operazione finale
assign result = (operation == 2'b00) ? and_out :
                (operation == 2'b01) ? or_out :
                (operation == 2'b10) ? sum :
                (operation == 2'b11) ? less : 1'b0;

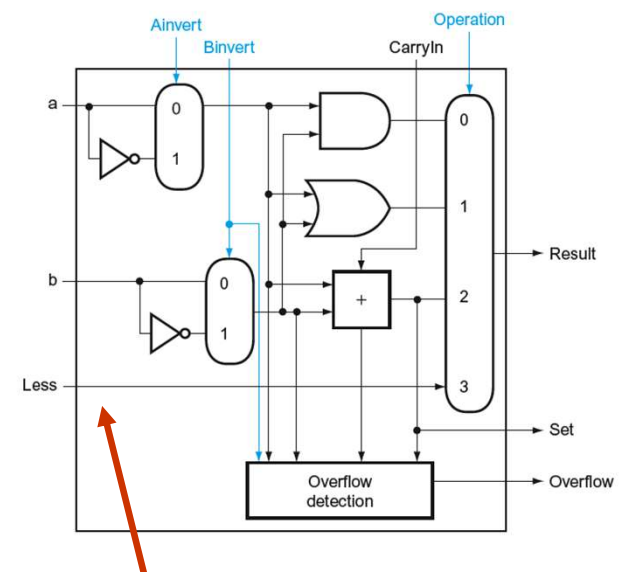
endmodule
```

RISC-V: ALU per slt

- L'ingresso Less dell'ALU per i bit da 1 a 31 deve essere uguale a 0
- L'ingresso Less dell'ALU per il bit 0 (il meno significativo) deve essere uguale al bit di segno del risultato della differenza $a - b$
 - ovvero il bit della somma dell'ALU del bit 31
- Per questo l'ALU per il bit 31 ha un'uscita Set per il risultato dell'addizionatore



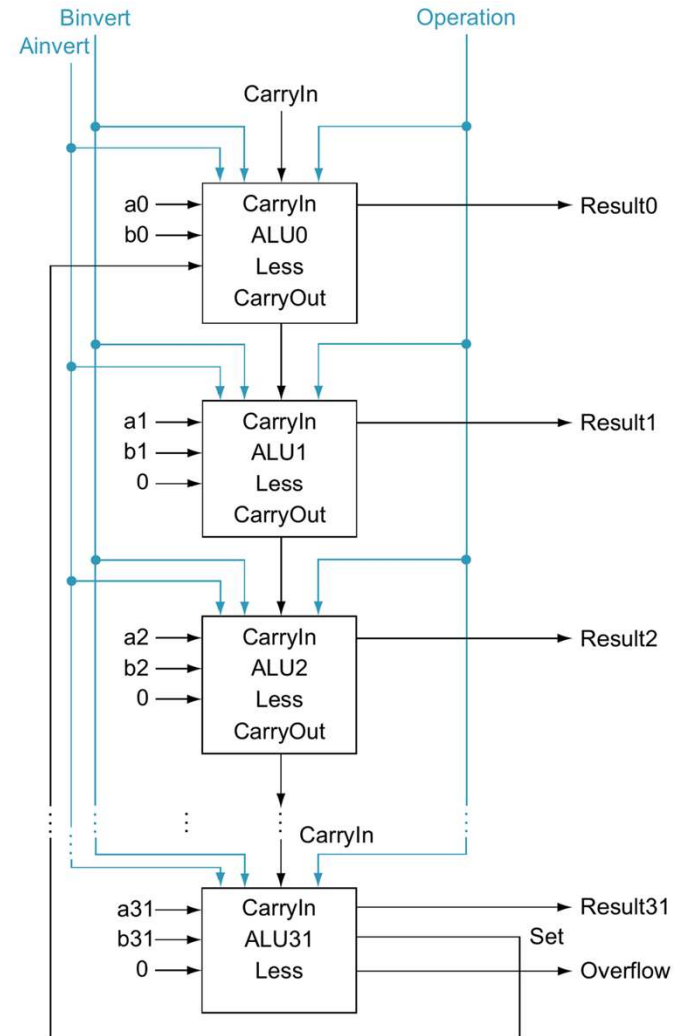
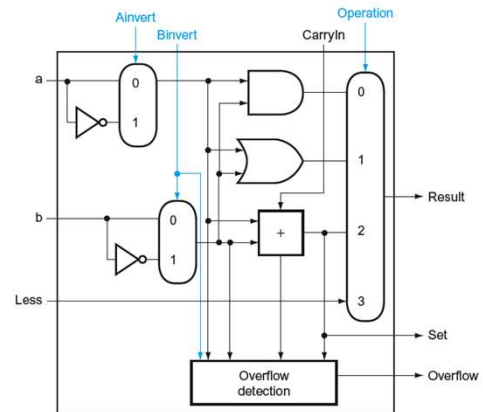
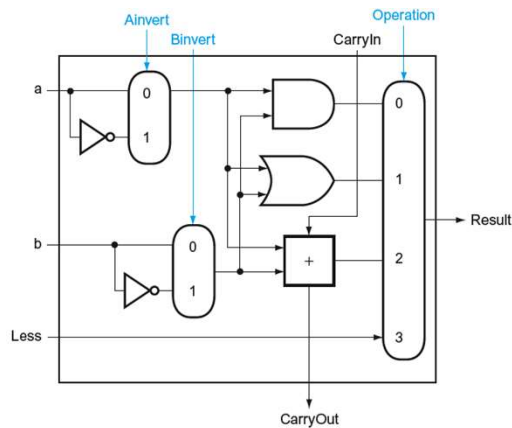
Ingresso a 0 per tutti tranne che per il bit meno significativo



Ingresso a 0

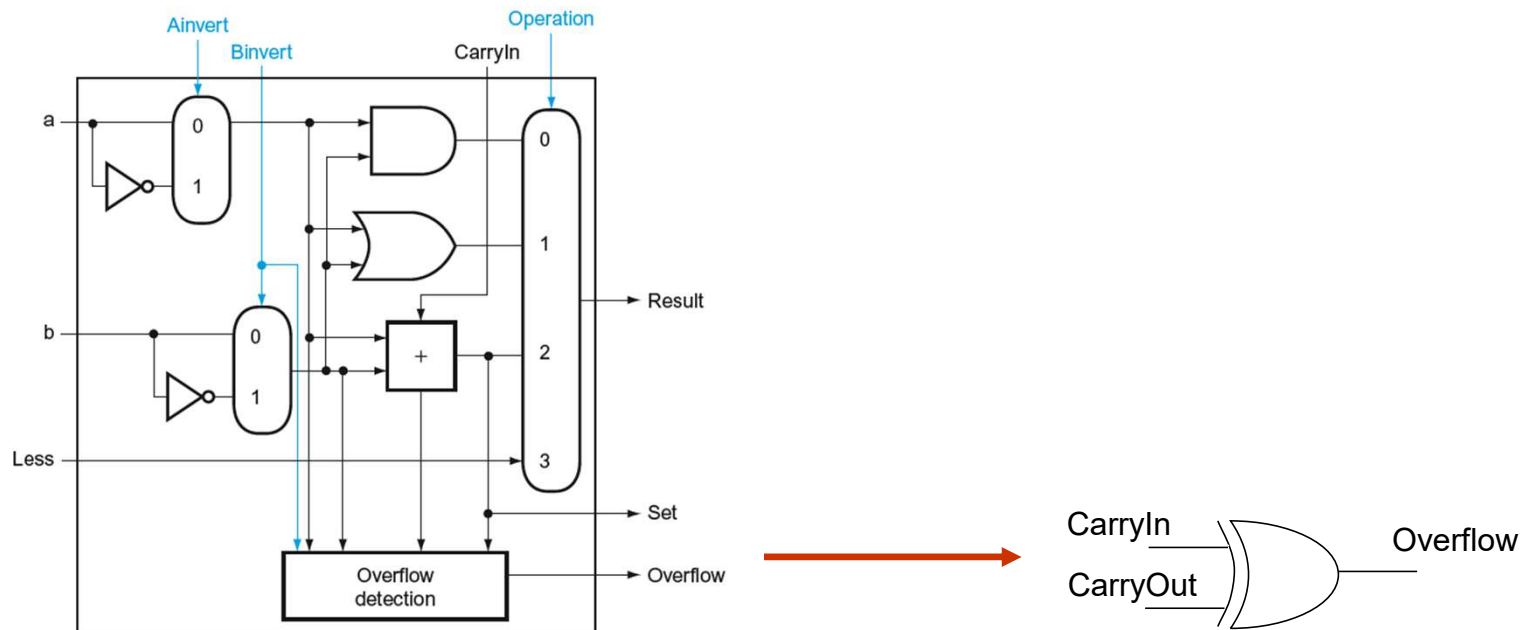
RISC-V: ALU per slt

Compressivamente



RISC-V: overflow

- Figura: Nella somma o differenza di due interi con segno in complemento a due, abbiamo overflow se i due operandi hanno lo stesso segno ma il risultato ha segno opposto.
- Ricordate la teoria del complemento a 2?
Si ha overflow quando il CarryIn ed il CarryOut del bit più significativo sono discordi.



Controllo operazioni dell'ALU

Ogni volta che vogliamo che l'ALU esegua sottrazioni dobbiamo asserire (porre a 1) sia CarryIn sia Binvert

Operazione	Funzione	Binvert	CarryIn0
0	AND	0	0
1	OR	0	0
2	ADD	0	0
2	SUB	1	1
3	SLT	1	1

CarryIn0 = Binvert

Usiamo una sola linea chiamata **Bnegate**

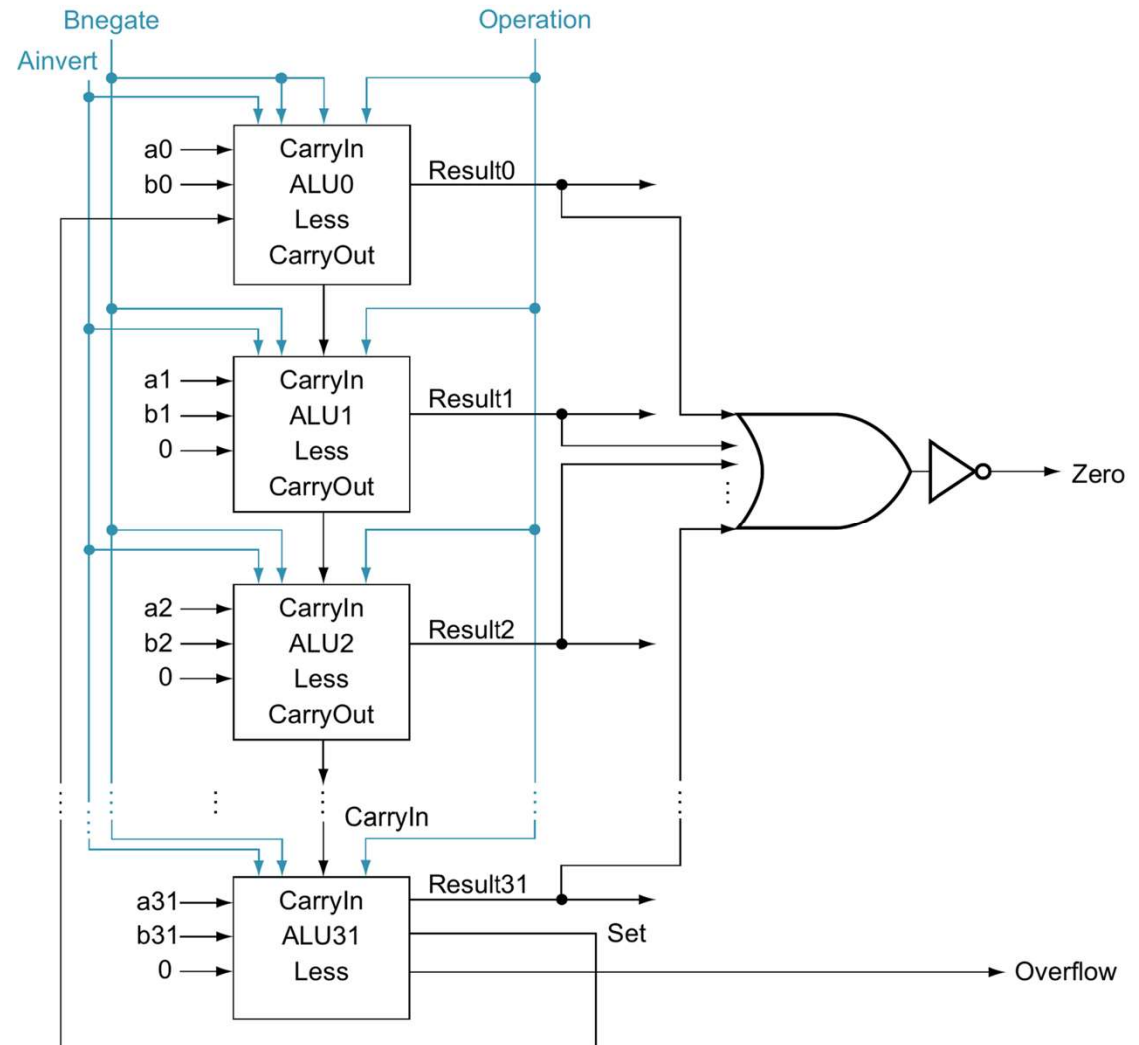
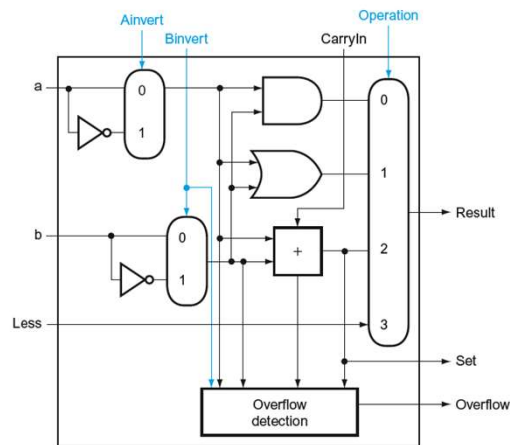
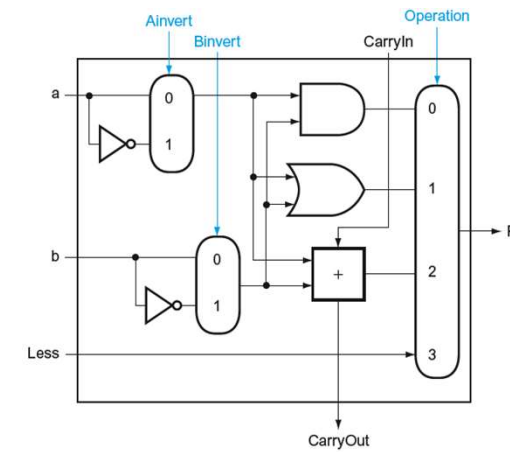
RISC-V: ALU per beq

- L'istruzione Branch if Equal (beq) realizza un salto se due registri sono uguali
- Si realizza il salto eseguendo $(a - b)$ e controllando se il risultato è uguale a 0
- L'ALU fornisce il valore 1 quando tutti i bit del risultato sono a zero, in quanto:

$$(a - b) == 0 \rightarrow a == b$$

L'uscita “Zero” vale 1 quando il risultato $(a-b)$ è zero!

RISC-V: ALU per beq



Controllo operazioni dell'ALU

- Per il controllo dell'ALU, possiamo pensare alla combinazione di
 - 1 bit per l'ingresso Ainvert
 - 1 bit per l'ingresso Bnegate
 - 2 bit per gli ingressi Operation
- come a linee di controllo a 4 bit per fare in modo che l'ALU esegua la somma, la sottrazione, l'AND, l'OR, la NOR o la slt

Ainvert	Bnegate	Operation
0	0	00
0	0	01
0	0	10
0	1	10
0	1	11
1	1	00

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set less than
1100	NOR

RISC-V: ALU

- Il simbolo comunemente usato per rappresentare una **ALU**
- Anche comunemente usato per indicare un **addizionatore**, quindi è prassi indicare esplicitamente con una scritta quale di questi due componenti si intende.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set less than
1100	NOR

